Davis & Shirtliff — Ultimate God Mode Architecture (Dynamic, Modular & Multi-Cloud)

Scaffolding-first · Repository contract · 3 mature tiers · Dynamic Targets · Dynamic Secrets · Multi-Cloud (AWS/GCP/Azure) · Concurrency-safe · External contributor flow · 24-hour approval window · Separated observability

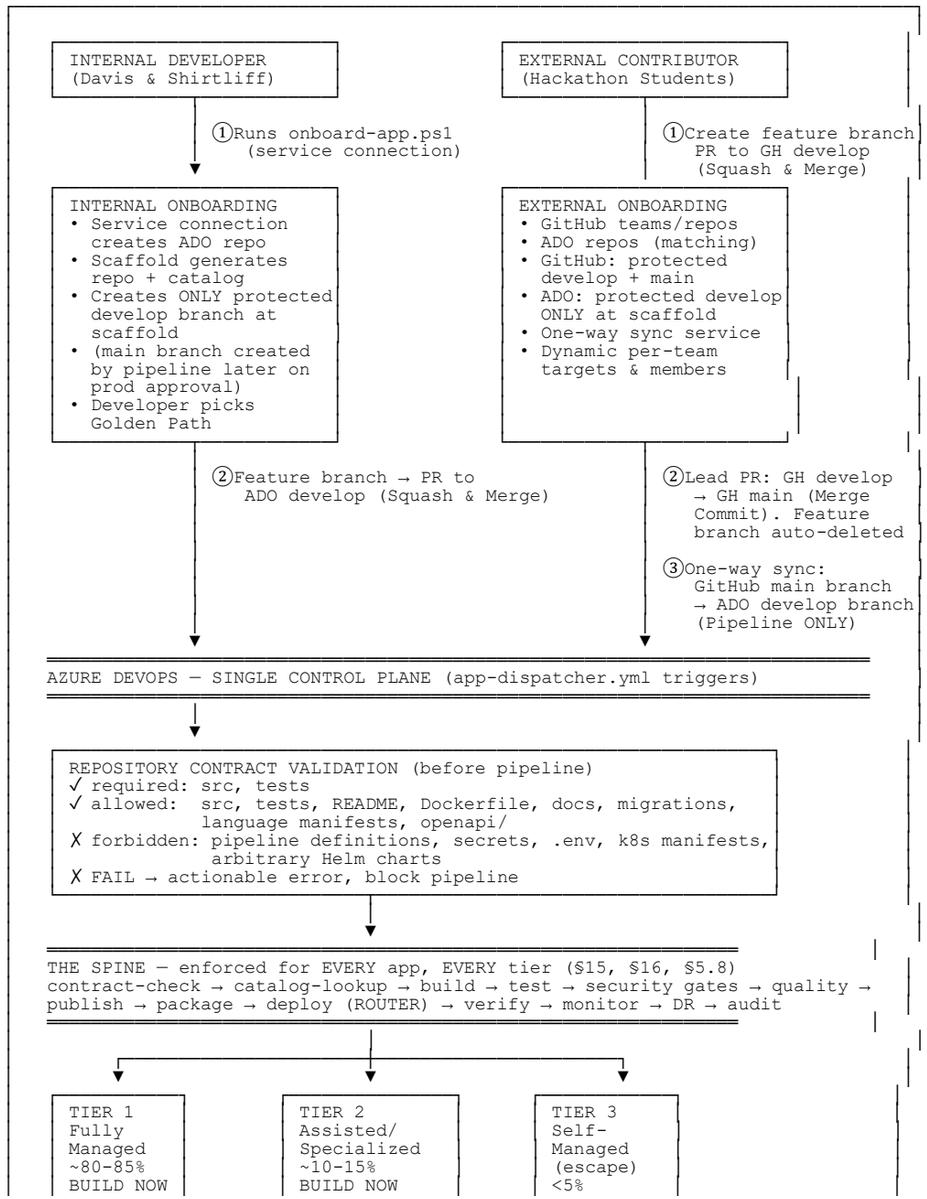
---

Corrections Applied to This Version (Expert Review + Multi-Cloud Expansions + Strict Git Workflows)

#	Correction	Before	After
1	**Branch model (ADO)**	Main branch created at scaffold time; direct pushes allowed	**Strict PR Workflow & Pipeline-driven ADO main**: Scaffold creates *only* the protected `develop` branch in ADO (no direct pushes). Developers use feature branches and **Squash & Merge** to `develop`. The `main` branch is created dynamically by the pipeline *after* staging approval. The pipeline uses **Create a merge commit** to move code from `develop` to `main`, then deploys the exact same staging image to production.
2	**Approval gate**	"24-hour auto-approval" (dangerous wording)	**"24-hour approval window with automatic cancellation on timeout"** — never interprets a timeout as approval
3	**Repo validation**	"Only src/ + tests/ allowed" (too restrictive)	**Repository contract**: required / allowed / forbidden lists. Allows docs/, migrations/, language manifests, openapi; forbids pipeline definitions, secrets, .env, k8s manifests, Helm charts
4	**Secrets model**	Hardcoded Azure Key Vault	**Dynamic Secrets Provider Abstraction**. Catalog declares `secretsProvider` (Azure KV, HashiCorp Vault, AWS Secrets Manager, CyberArk Conjur, Akeyless). Platform routes to correct provider module
5	**DR model**	"Data + VM replicas" (incomplete)	Three explicit layers: **High Availability** (replicas/zones), **Disaster Recovery** (secondary region + geo-replication), **Cyber-resilience** (immutable backups, isolated access, quarterly recovery tests)
6	**Developer experience**	PowerShell forever	**Evolution path**: PowerShell (Phase 1) → Platform API (Phase 2) → Developer Portal (Phase 3 end-state)
7	**Observability**	Grafana as sole source of truth	**Three separated concerns**: Operational observability (Grafana/Prometheus/Cloud Monitors), Developer

			feedback (ADO PR checks/notifications), Security governance (Defender/Sentinel/Policy)
8	**Deployment Targets**	Hardcoded Azure AKS	**Dynamic Deployment Target Abstraction**. Catalog declares `deploymentTarget` (AKS, EKS, GKE, DigitalOcean, Firebase, Supabase, AWS Amplify, Kamatera, Hostinger, etc.). Supports K8s, PaaS, BaaS, and VMs
9	**Non-K8s Workloads**	Assumed Kubernetes/Helm	**Provider-agnostic deploy step**. Pipeline routes to native CLIs/APIs (`firebase deploy`, `supabase db push`, `doctl`, AWS CDK) based on target. Mobile apps and static sites fully supported
10	**Extensibility**	Monolithic Terraform/Helm	**Modular Provider Plugin Architecture**. New tech stack = add folder to `providers/compute/` or `providers/secrets/`. Core Spine never changes. Freedom of choice for any tech stack
11	**Branch model (GitHub)**	Assumed identical to ADO	**Strict External Workflow**: Scaffold creates protected `develop` and `main` in GitHub. Students use feature branches, **Squash & Merge** to `develop`. Student Lead uses **Create a merge commit** from `develop` to `main`. Head branches auto-delete. ADO `develop` is strictly protected; only the sync service can write to it.
12	**Multi-Cloud Routing**	Implicit cloud support	**Explicit Multi-Cloud Routing & Security**. AWS EKS (IRSA), GCP GKE (Workload Identity), and Azure AKS (Entra WI) are fully supported via identical pipeline spine and dynamic provider plugins
13	**Dynamic Observability**	Hardcoded Azure Monitor	**Dynamic Observability Provider**. Catalog declares `observabilityProvider` (Azure Monitor, CloudWatch, Cloud Logging, Datadog, etc.) routing metrics to the correct cloud-native or 3rd-party stack
14	**Target Environments**	Prod/DR only in registry	**Staging, Prod, and DR** explicitly mapped for all clouds. **Kubernetes is strictly optional** per cloud (AWS App Runner, GCP Cloud Run, Azure App Service supported alongside EKS/GKE/AKS)
15	**Hackathon Inputs**	Fixed team sizes & uniform defaults	**Dynamic Hackathon Configuration**: Team member counts vary per team (e.g., 3, 4, 6). Target, secrets, and observability providers are chosen *per team*, not applied as a single global default.
16	**Git Merge Strategies**	Undefined	Explicitly enforced: **Squash & Merge** for feature → `develop`. **Create a merge commit** for `develop` → `main` (both GitHub Lead and ADO Pipeline). **Auto-delete head branches** enabled in GitHub.

DIAGRAM 1 — The North Star (Total System: Internal + External + Multi-Cloud)



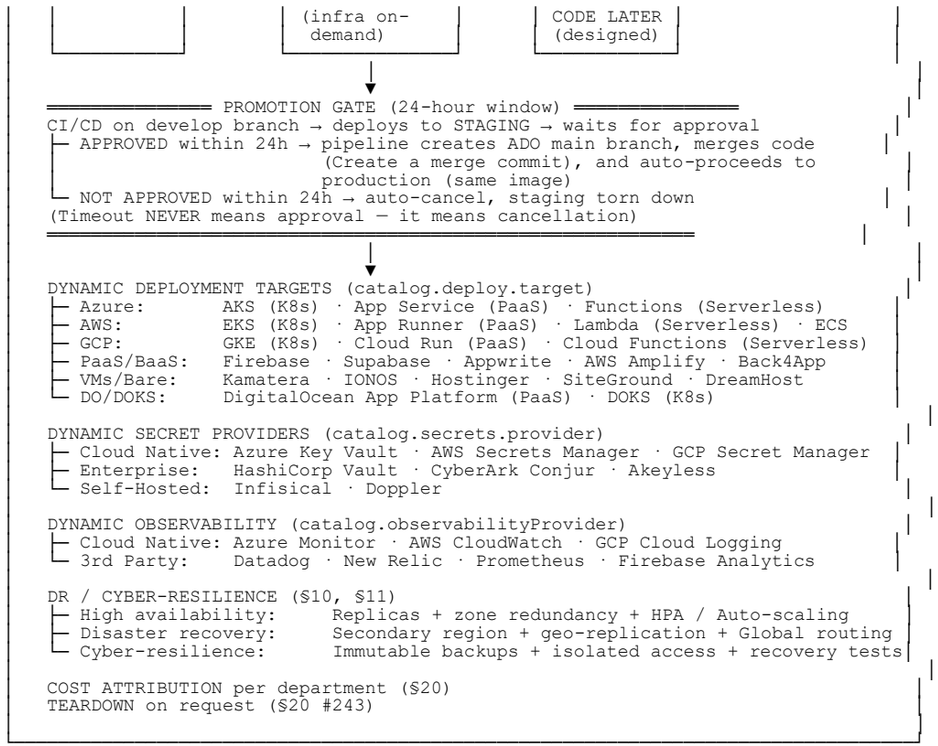


DIAGRAM 2 — Complete Repository Structure (Internal + External, Modular Providers)

```
D&S Azure DevOps (Organization)
│
├── □ PROJECT: ds-platform                          [Platform team — the engine]
│   │
│   ├── □ REPO: onboarding
│   │   ├── onboard-app.ps1                         ◄── INTERNAL onboarding (PowerShell)
│   │   ├── onboard-hackathon.ps1                   ◄── EXTERNAL onboarding (PowerShell)
│   │   ├── register-custom-app.ps1                 ◄── Tier 3 registration (later)
│   │   ├── terraform/                              ◄── Terraform modules for repo creation
│   │   │   ├── ado-repo/                           ◄── Creates ADO repos (service connection)
│   │   │   ├── github-repo/                        ◄── Creates GitHub repos/teams (service connection)
│   │   │   └── variables.tf                        ◄── Editable variables (modular)
│   │   └── lib/
│   │       ├── entra-auth.ps1                      ◄── Auth + department constraint
│   │       ├── scaffold.ps1                        ◄── Generates repo + catalog
│   │       ├── validate-contract.ps1               ◄── Repository contract validation
│   │       └── submit-prs.ps1                      ◄── Raises PRs to app-catalog
```


REPO: golden-path-templates ← THE FRONT DOOR (scaffolding)
_catalog.json ← Menu + descriptions + decision guide
web-api/ · mobile-baas/ · data-bi/ · business-central/
infrastructure/ · peripheral-software/ ← TIER 1 — standard paths
specialized/ ← TIER 2 — specialized paths
ai-ml-training/ · streaming/ · edge/
REPO: pipeline-templates
app-dispatcher.yml ← Master pipeline (all apps extend)
platform-policy.yml ← THE BRAIN (derivation rules)
_shared/ ← THE SPINE (all tiers)
repo-contract-check.yml ← Validates required/allowed/forbidden
catalog-lookup.yml ← Reads catalog (goldenPath known)
ci-base.yml ← Build + tests
devsecops-gates.yml ← SAST/SCA/secret/container/SBOM/IaC
quality-gates.yml ← Coverage/smells/license
publish-results.yml ← → Prometheus/Log Analytics → Grafana
package.yml ← Build artifact (image, bundle, zip)
deploy-router.yml ← ROUTES to correct provider module
verify.yml ← Health checks + smoke tests
monitor-register.yml ← Register Observability Provider
promote-to-main.yml ← Creates ADO main + Merge Commit
dr-provision.yml ← Terraform HA + DR + cyber-resilience
dr-teardown.yml ← Destroy replicas on request
golden-paths/ ← Deploy profiles
web-api.yml · mobile-baas.yml · data-bi.yml
business-central.yml · infrastructure.yml · peripheral-software.yml
specialized/
ai-ml-training.yml · streaming.yml · edge.yml
escape-hatch/ ← TIER 3 (code later)
escape-hatch.yml ← Trigger team pipeline + enforce spine
contract-verify.yml ← Verify results/observability/cost
REPO: providers ← THE EXTENSIBILITY ENGINE (Modular)
compute/ ← Deployment Targets
azure-aks/ ← Terraform + Helm (K8s)
deploy.yml ← Helm + CSI driver
workload-identity.yml ← Entra federated credential
terraform/ ← AKS cluster provisioning
azure-app-service/ ← Azure CLI / Bicep (PaaS, NO K8s)
azure-functions/ ← Azure CLI (Serverless, NO K8s)
aws-eks/ ← AWS PLUGIN (K8s)
deploy.yml ← Helm + EKS CSI driver
iam-role-for-service-accounts.yml ← IRSA setup
terraform/
main.tf ← EKS cluster module
irsa.tf ← OIDC provider + IAM roles
variables.tf
README.md
aws-app-runner/ ← AWS CLI / CDK (PaaS, NO K8s)
aws-lambda/ ← AWS SAM / CDK (Serverless, NO K8s)
gcp-gke/ ← GCP PLUGIN (K8s)
deploy.yml ← Helm + GKE CSI driver
workload-identity.yml ← GKE Workload Identity
terraform/
main.tf ← GKE cluster module
workload-identity.tf ← GSA ↔ KSA binding
variables.tf
README.md
gcp-cloud-run/ ← gcloud CLI (PaaS, NO K8s)
gcp-cloud-functions/ ← gcloud CLI (Serverless, NO K8s)
digitalocean-app/ ← Terraform + doctl (PaaS, NO K8s)
digitalocean-doks/ ← Terraform + Helm (K8s)
firebase/ ← Terraform + firebase CLI (BaaS)
supabase/ ← Terraform + supabase CLI (BaaS)
aws-amplify/ ← Terraform + AWS CDK (PaaS)
kamatera-vm/ ← Terraform + SSH/Ansible (VM)
ionos-vm/ · hostinger/ · siteground/ ← Any provider you add
... (modular, add any tech stack)
secrets/ ← Secret Providers
azure-key-vault/ ← CSI driver + Workload Identity

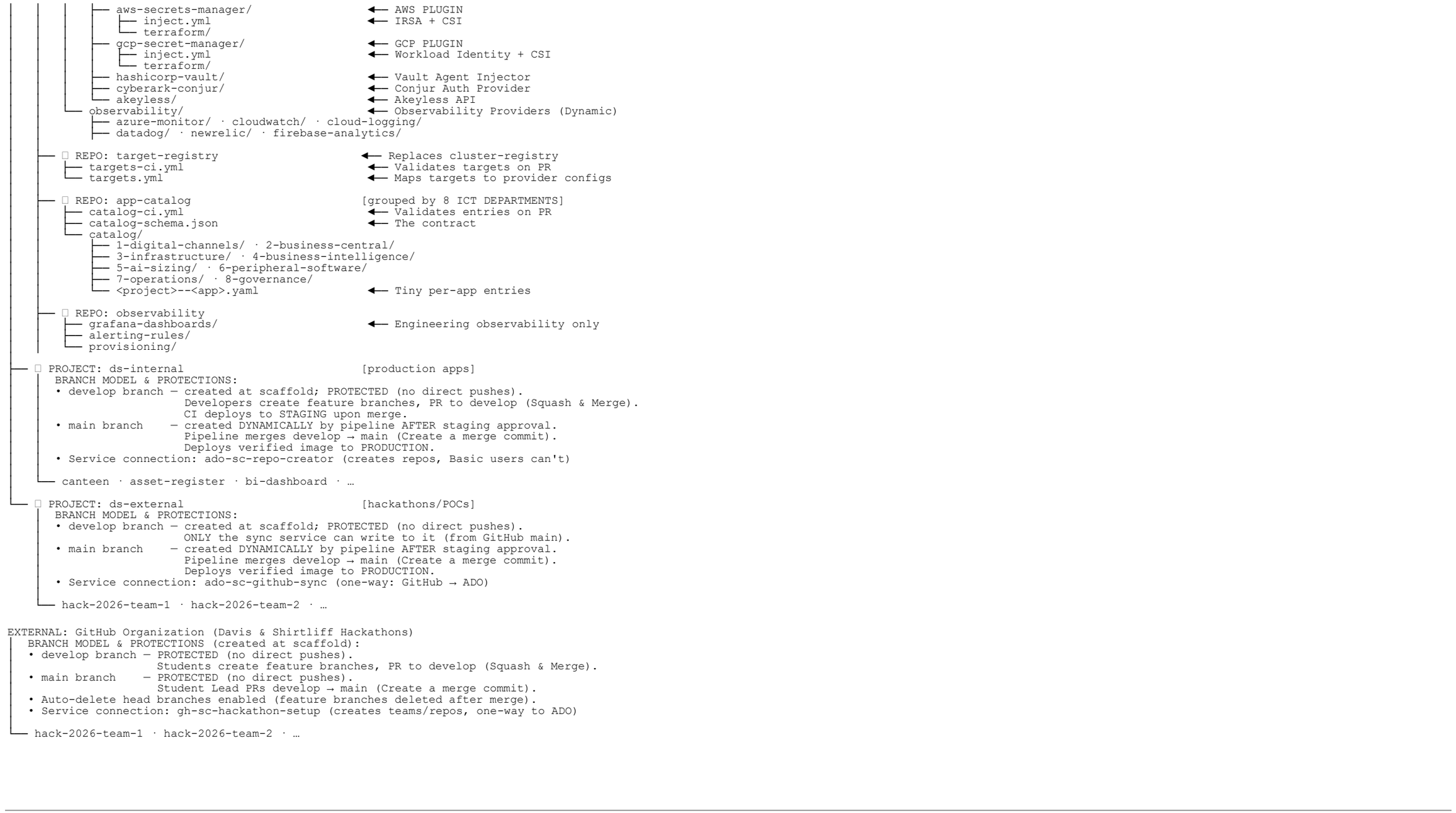
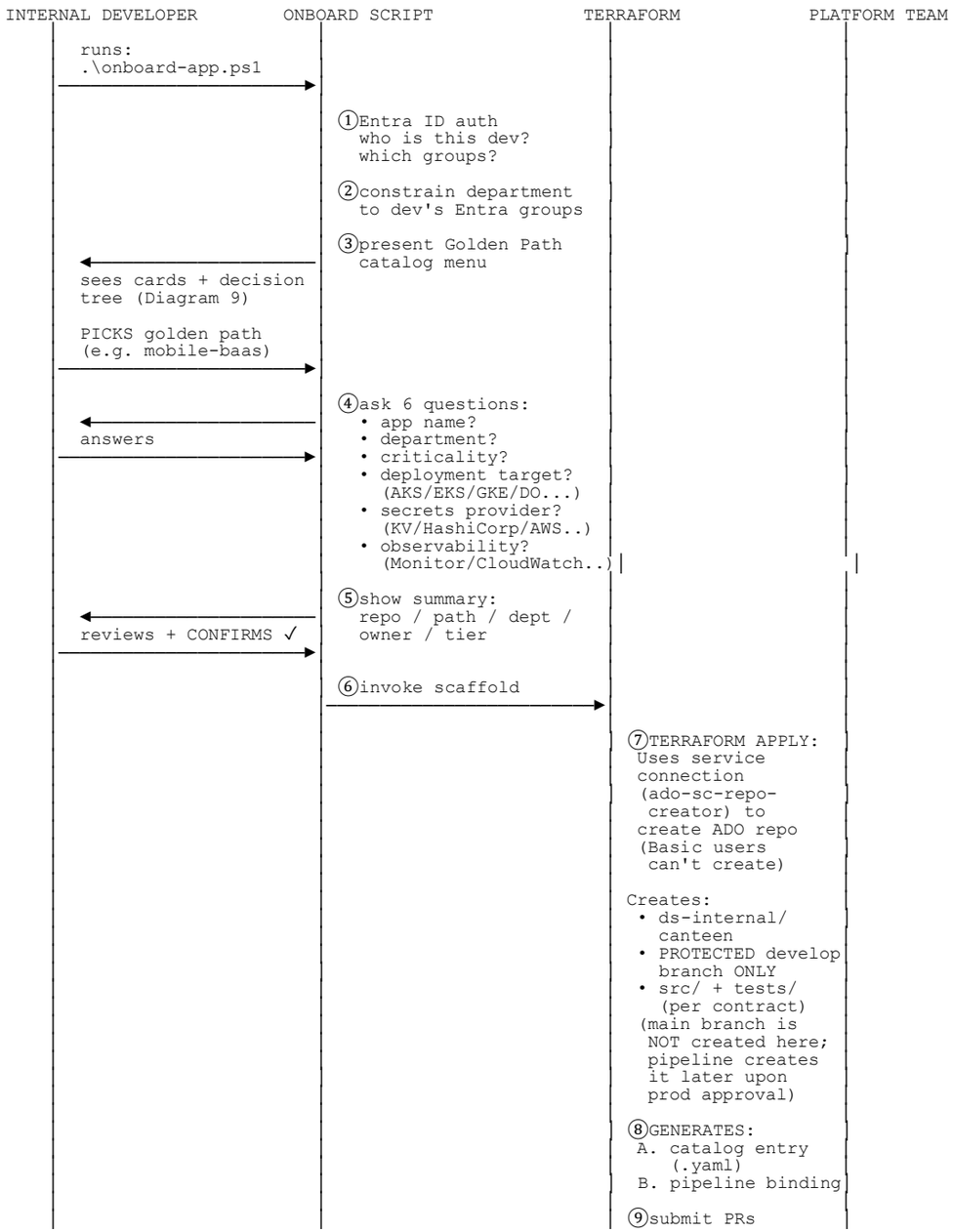
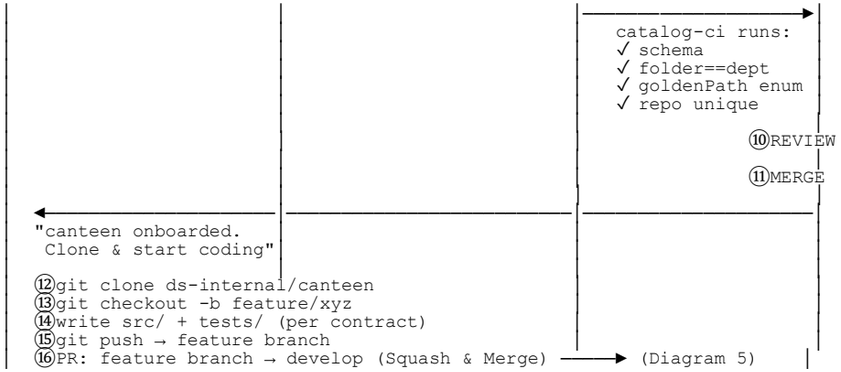

├── aws-secrets-manager/ ◄── AWS PLUGIN
├── inject.yml ◄── IRSA + CSI
└── terraform/
├── gcp-secret-manager/ ◄── GCP PLUGIN
├── inject.yml ◄── Workload Identity + CSI
└── terraform/
├── hashicorp-vault/ ◄── Vault Agent Injector
├── cyberark-conjur/ ◄── Conjur Auth Provider
└── akeyless/ ◄── Akeyless API
└── observability/ ◄── Observability Providers (Dynamic)
├── azure-monitor/ · cloudwatch/ · cloud-logging/
├── datadog/ · newrelic/ · firebase-analytics/
├── REPO: target-registry ◄── Replaces cluster-registry
├── targets-ci.yml ◄── Validates targets on PR
└── targets.yml ◄── Maps targets to provider configs
├── REPO: app-catalog [grouped by 8 ICT DEPARTMENTS]
├── catalog-ci.yml ◄── Validates entries on PR
├── catalog-schema.json ◄── The contract
└── catalog/
├── 1-digital-channels/ · 2-business-central/
├── 3-infrastructure/ · 4-business-intelligence/
├── 5-ai-sizing/ · 6-peripheral-software/
├── 7-operations/ · 8-governance/
└── <project>--<app>.yaml ◄── Tiny per-app entries
├── REPO: observability
├── grafana-dashboards/ ◄── Engineering observability only
├── alerting-rules/
└── provisioning/
├── PROJECT: ds-internal [production apps]
BRANCH MODEL & PROTECTIONS:
• develop branch — created at scaffold; PROTECTED (no direct pushes).
Developers create feature branches, PR to develop (Squash & Merge).
CI deploys to STAGING upon merge.
• main branch — created DYNAMICALLY by pipeline AFTER staging approval.
Pipeline merges develop → main (Create a merge commit).
Deploys verified image to PRODUCTION.
• Service connection: ado-sc-repo-creator (creates repos, Basic users can't)
└── canteen · asset-register · bi-dashboard · …
└── PROJECT: ds-external [hackathons/POCs]
BRANCH MODEL & PROTECTIONS:
• develop branch — created at scaffold; PROTECTED (no direct pushes).
ONLY the sync service can write to it (from GitHub main).
• main branch — created DYNAMICALLY by pipeline AFTER staging approval.
Pipeline merges develop → main (Create a merge commit).
Deploys verified image to PRODUCTION.
• Service connection: ado-sc-github-sync (one-way: GitHub → ADO)
└── hack-2026-team-1 · hack-2026-team-2 · …
EXTERNAL: GitHub Organization (Davis & Shirtliff Hackathons)
BRANCH MODEL & PROTECTIONS (created at scaffold):
• develop branch — PROTECTED (no direct pushes).
Students create feature branches, PR to develop (Squash & Merge).
• main branch — PROTECTED (no direct pushes).
Student Lead PRs develop → main (Create a merge commit).
• Auto-delete head branches enabled (feature branches deleted after merge).
• Service connection: gh-sc-hackathon-setup (creates teams/repos, one-way to ADO)
└── hack-2026-team-1 · hack-2026-team-2 · …

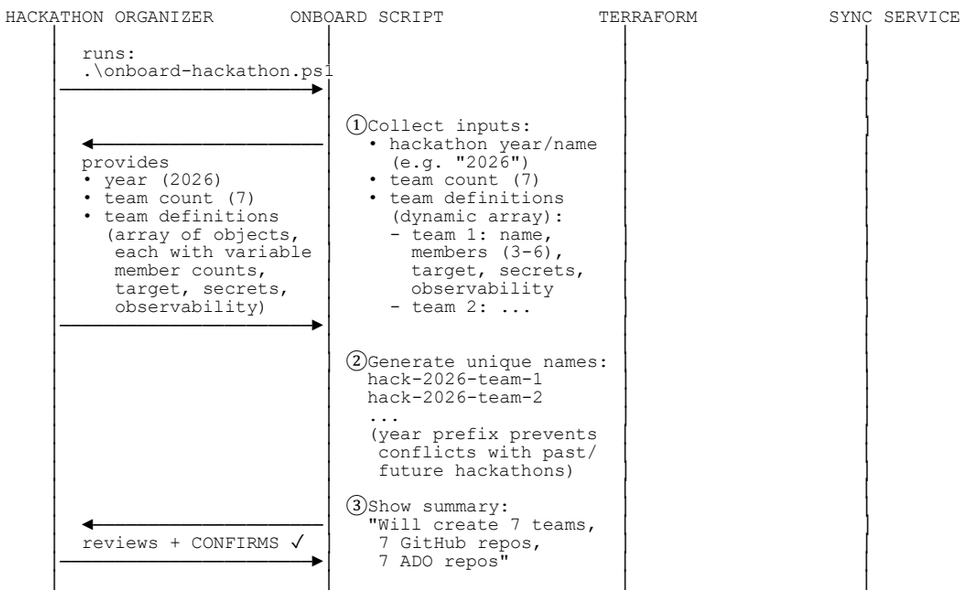
DIAGRAM 3 — Internal Onboarding Flow (Service Connection, Scaffold)

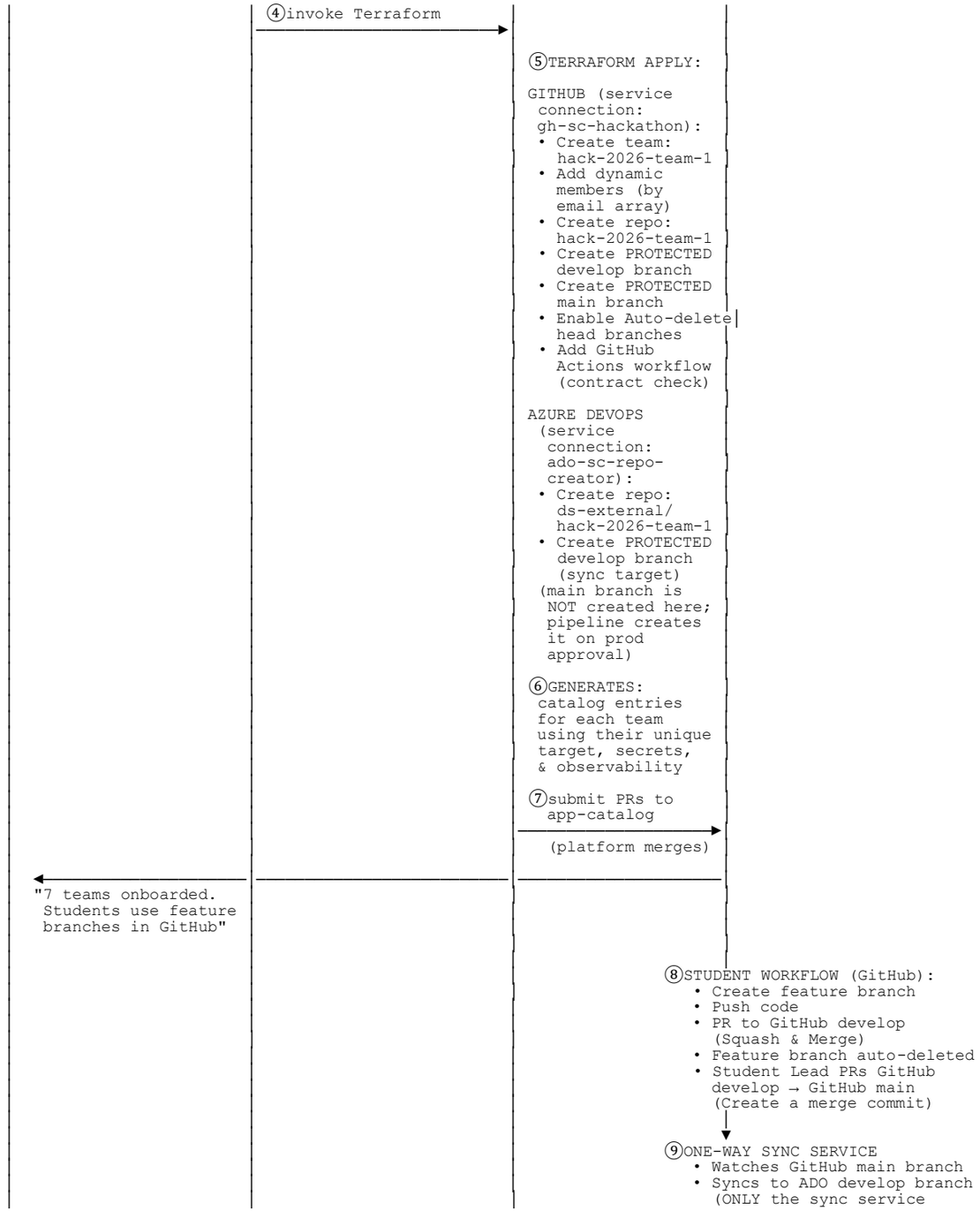




Key: Service connection creates the repo (developer has Basic license). ONLY the protected `develop` branch is created at scaffold time in ADO (no direct pushes allowed). Developers must use feature branches and Squash & Merge. The `main` branch is created dynamically by the pipeline later, after staging deployment and approval.

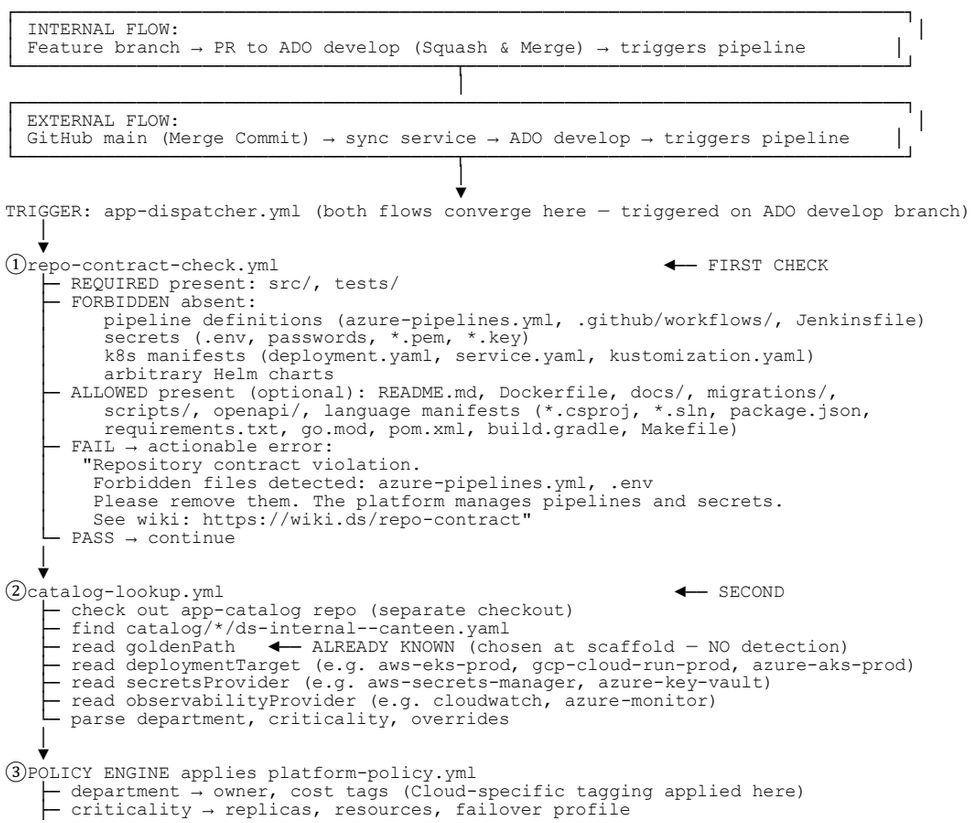
DIAGRAM 4 — External Onboarding Flow (Hackathon: GitHub + ADO)




④invoke Terraform
⑤TERRAFORM APPLY:
GITHUB (service connection: gh-sc-hackathon):
• Create team: hack-2026-team-1
• Add dynamic members (by email array)
• Create repo: hack-2026-team-1
• Create PROTECTED develop branch
• Create PROTECTED main branch
• Enable Auto-delete head branches
• Add GitHub Actions workflow (contract check)
AZURE DEVOPS (service connection: ado-sc-repo-creator):
• Create repo: ds-external/ hack-2026-team-1
• Create PROTECTED develop branch (sync target)
(main branch is NOT created here; pipeline creates it on prod approval)
⑥GENERATES: catalog entries for each team using their unique target, secrets, & observability
⑦submit PRs to app-catalog
(platform merges)
"7 teams onboarded. Students use feature branches in GitHub"
⑧STUDENT WORKFLOW (GitHub):
• Create feature branch
• Push code
• PR to GitHub develop (Squash & Merge)
• Feature branch auto-deleted
• Student Lead PRs GitHub develop → GitHub main (Create a merge commit)
⑨ONE-WAY SYNC SERVICE
• Watches GitHub main branch
• Syncs to ADO develop branch
(ONLY the sync service

Key: External onboarding creates both GitHub and ADO repos with matching names. GitHub gets protected `develop` and `main` branches at scaffold, with auto-delete head branches enabled. ADO gets *only* the protected `develop` branch at scaffold (writable ONLY by the sync service). Team sizes, targets, secrets, and observability are fully dynamic per team.

DIAGRAM 5 — Workflow from `git push` to Production (Internal + External + Multi-Cloud)



```
   └─ goldenPath → template library, deploy profile
   │
   ▼
④ROUTE on goldenPath (runtime condition)
   │  web-api/mobile-baas/data-bi/business-central/infrastructure/peripheral
   │  (specialized/* for Tier 2 · "custom" for Tier 3)
   ▼
⑤ci-base.yml — build + unit/integration tests              [BUILD → TEST]
   │  If code conflicts with the chosen path, BUILD fails with
   │  an actionable error (§258) — the safety net, not detection.
   ▼
⑥devsecops-gates.yml + quality-gates.yml                   [SECURITY/QUALITY]
   ├─ SAST · SCA · secret · container · IaC · SBOM · coverage
   ├─ FAIL → block + actionable feedback (§252)
   └─ PASS → continue
   │
   ▼
⑦publish-results.yml → Prometheus/Log Analytics → Grafana   [→ OBSERVABILITY]
   │  (security-posture, devsecops-gates, quality, vulnerabilities)
   │  ALSO: writes structured summary to ADO Work Items + PR comments (dev feedback)
   │  ALSO: exports to Microsoft Defender for DevOps (governance)
   ▼
⑧package.yml — build artifact ONCE (tag = build id)       [PACKAGE]
   │  • IF K8s target: builds Docker image → ACR/ECR/GCR
   │  • IF PaaS target (App Runner/Cloud Run): builds Docker image → ECR/GCR
   │  • IF BaaS target: builds static bundle / serverless zip
   │  • Artifact contains NO secrets (Dynamic Secrets default)
   ▼
⑨deploy-router.yml → STAGING environment                   [DEPLOY TO STAGING]
   │  reads catalog.deploy.target, catalog.secrets.provider, catalog.observabilityProvider
   │
   │  HOW THE ROUTING WORKS (Same Pipeline, Different Cloud):
   │  Pipeline Spine (UNCHANGED) → deploy-router.yml (branches) →
   │
   │  ┌──────┬──────┬──────┬──────┬──────┬──────┬──────┬──────┐
   │  ▼      ▼      ▼      ▼      ▼      ▼      ▼      ▼      ▼
   │ AKS    EKS    GKE   App Svc App Run Cloud Run DO App Firebase
   │ plugin plugin plugin plugin plugin plugin   plugin  plugin
   │
   │  THE SECURITY MODEL PER CLOUD (All Workload-Identity Based where applicable):
   │
   │  ┌──────────────┬──────────────────────────┬──────────────────────────────────┐
   │  │ Cloud        │ Identity Mechanism       │ How it works                     │
   │  ├──────────────┼──────────────────────────┼──────────────────────────────────┤
   │  │ Azure AKS    │ Workload Identity (Entra)│ ServiceAccount ↔ Entra App (fed cred)
   │  │ Azure PaaS   │ Managed Identity         │ App Service ↔ Entra ID           │
   │  │ AWS EKS      │ IRSA (IAM Roles for SAs) │ ServiceAccount ↔ IAM Role via OIDC
   │  │ AWS PaaS     │ IAM Execution Role       │ App Runner ↔ IAM Role            │
   │  │ GCP GKE      │ Workload Identity (GCP)  │ KSA ↔ GSA binding                │
   │  │ GCP PaaS     │ Service Account          │ Cloud Run ↔ GCP SA               │
   │  └──────────────┴──────────────────────────┴──────────────────────────────────┘
   │
   ├─ IF target == azure-aks:
   │     calls providers/compute/azure-aks/deploy.yml (Helm + CSI + Workload Identity)
   ├─ IF target == azure-app-service:
   │     calls providers/compute/azure-app-service/deploy.yml (az webapp up + Managed Identity)
   ├─ IF target == aws-eks:
   │     calls providers/compute/aws-eks/deploy.yml (Helm + CSI + IRSA)
   ├─ IF target == aws-app-runner:
   │     calls providers/compute/aws-app-runner/deploy.yml (aws apprunner + IAM Role)
   ├─ IF target == gcp-gke:
   │     calls providers/compute/gcp-gke/deploy.yml (Helm + CSI + GKE WI)
   ├─ IF target == gcp-cloud-run:
   │     calls providers/compute/gcp-cloud-run/deploy.yml (gcloud run deploy + SA)
   ├─ IF target == digitalocean-app-platform:
   │     calls providers/compute/digitalocean-app/deploy.yml (doctl)
   ├─ IF target == firebase:
   │     calls providers/compute/firebase/deploy.yml (firebase deploy)
   ├─ IF target == supabase:
   │     calls providers/compute/supabase/deploy.yml (supabase db push)
   │
   │  AND routes secrets:
   ├─ IF secrets == azure-key-vault:
   │     calls providers/secrets/azure-key-vault/inject.yml (CSI + WI or Managed ID)
   ├─ IF secrets == aws-secrets-manager:
   │     calls providers/secrets/aws-secrets-manager/inject.yml (IRSA + CSI or IAM Role)
   ├─ IF secrets == gcp-secret-manager:
```


calls providers/secrets/gcp-secret-manager/inject.yml (GKE WI + CSI or SA)
IF secrets == hashicorp-vault:
calls providers/secrets/hashicorp-vault/inject.yml (Vault Agent)
AND routes observability:
IF observability == azure-monitor: calls providers/observability/azure-monitor/
IF observability == cloudwatch: calls providers/observability/cloudwatch/
IF observability == cloud-logging: calls providers/observability/cloud-logging/
⑩verify.yml — health checks + smoke tests [VERIFY]
⑪monitor-register.yml → Observability Provider [MONITOR]
⑫PROMOTION GATE (24-hour window) [WAIT FOR APPROVAL]
• Staging is live. Approvers notified via email/Teams.
• Pipeline pauses, waiting for manual approval gate.
PROMOTION GATE (24-hour window)
Approval required (email/users notified)
APPROVED within 24 hours:
⑬Pipeline creates ADO main branch (if it doesn't exist),
merges ADO develop → ADO main (Create a merge commit).
Triggers production pipeline:
deploy-router.yml → PROD environment (SAME artifact from staging, not rebuilt)
⑭dr-provision.yml — Terraform HA + DR + cyber-resilience [RESILIENCE]
High Availability: Replicas + zone redundancy + Auto-scaling
Disaster Recovery: secondary region + geo-replication + Global routing
(Can target completely different cloud, e.g. Azure Prod → AWS DR)
Cyber-resilience: immutable backups (WORM) in isolated storage
+ separate break-glass admin + quarterly recovery test
(idempotent · prod only · reads catalog.resilience)
NOT APPROVED within 24 hours:
AUTO-CANCEL + STAGING TEARDOWN
Pipeline aborted with comment: "Approval window expired"
staging deployment torn down (cost hygiene)
notification sent: "Approval cancelled — reopen when ready"
developer must re-trigger (never silently deploys to prod)
RUNTIME OBSERVABILITY (continuous, independent of CI/CD — §9)
OPERATIONAL: Cloud Monitors + Prometheus + App Insights → Grafana dashboards
DEVELOPER: ADO PR checks + pipeline summary + Teams notifications
GOVERNANCE: Microsoft Defender for DevOps + Sentinel + Cloud Policy + Audit logs
failure detected → diagnostics auto-collected
→ alert routed to app owner (not Gregory!)
→ actionable message:
"Deployment failed.
Application: Team 7 API
Environment: production
Target: AWS App Runner (eu-west-1)
Failure: Health check failed
Probable cause: missing AWS Secrets Manager secret
Evidence: deployment event log
Recommended action: verify secret 'canteen-aws/db-password' exists in AWS SM
Owner: Team 7
Platform engineer intervention: not required"
[on request] dr-teardown.yml — destroy replicas/sandbox [COST LIFECYCLE §20]

Key Changes in This Diagram:

1. Strict Git Workflows: Internal flow explicitly requires PR to `develop` (Squash & Merge). External flow explicitly requires sync from GitHub `main` (Merge Commit) to ADO `develop`.

2. Pipeline-Driven ADO Main Branch: The ADO `main` branch is NOT created at scaffold. It is created by the pipeline in Step ⑬ *only after* staging verification and manual approval. Pipeline merges `develop` to `main` using **Create a merge commit**.

3. Image Promotion: The exact same image verified in staging is deployed to production.

4. Repository contract (required/allowed/forbidden) replaces strict "src/+tests/ only"

5. 24-hour approval window with auto-cancel on timeout (never auto-approves)

6. Dynamic Targets, Secrets & Observability explicitly routed in Step ⑨ (deploy-router.yml), now including AWS EKS and GCP GKE

7. Multi-Cloud Routing Mechanics & Security Models (IRSA, GKE WI, PaaS Managed Identities) explicitly detailed in Step ⑨

8. Kubernetes is strictly optional: PaaS and Serverless routing options provided for AWS, GCP, Azure, and DO

9. Non-K8s workloads fully supported via provider-specific CLIs/APIs

10. DR expanded to three layers: HA, Disaster Recovery (cross-cloud supported), Cyber-resilience

11. Observability separated into three concerns (operational/developer/governance)

12. Failure diagnostics give actionable info to app owner (not platform team)

DIAGRAM 6 — Catalog `.yaml` for Each Tier (Dynamic Providers & Multi-Cloud)

☁ Tier 1 — Azure AKS + Azure Key Vault + Azure Monitor

```
# catalog/6-peripheral-software/ds-internal--canteen-azure.yaml
department: peripheral-software
criticality: high
goldenPath: web-api
deploymentTarget: azure-aks-prod          # ← Azure K8s
secretsProvider: azure-key-vault          # ← Azure
observabilityProvider: azure-monitor      # ← Dynamic Observability

config:
  ingress: { host: canteen.dayliff.com }
  database: { type: azure-sql, sku: S1 }
secrets:
  - canteen-db-password
  - canteen-jwt-signing-key
```

☁ Tier 1 — AWS App Runner (NO Kubernetes) + AWS Secrets Manager + CloudWatch

```
# catalog/6-peripheral-software/ds-internal--canteen-aws-paas.yaml
department: peripheral-software
```

```
criticality: high
goldenPath: web-api
deploymentTarget: aws-app-runner-prod   # ← AWS PaaS (NO K8s required)
secretsProvider: aws-secrets-manager    # ← AWS
observabilityProvider: cloudwatch       # ← Dynamic Observability

config:
  domain: canteen-aws-paas.dayliff.com
  instanceType: cpu-1-mem-2
secrets:
  - canteen-aws/db-password
  - canteen-aws/jwt-signing-key
```

☁ Tier 1 — GCP Cloud Run (NO Kubernetes) + GCP Secret Manager + Cloud Logging

```
# catalog/6-peripheral-software/ds-internal--canteen-gcp-paas.yaml
department: peripheral-software
criticality: high
goldenPath: web-api
deploymentTarget: gcp-cloud-run-prod    # ← GCP PaaS (NO K8s required)
secretsProvider: gcp-secret-manager     # ← GCP
observabilityProvider: cloud-logging    # ← Dynamic Observability

config:
  domain: canteen-gcp-paas.dayliff.com
  concurrency: 80
secrets:
  - projects/ds-prod/secrets/canteen-db-password
  - projects/ds-prod/secrets/canteen-jwt-signing-key
```

☁ Tier 1 — Azure App Service (NO Kubernetes)

```
# catalog/6-peripheral-software/ds-internal--canteen-azure-paas.yaml
department: peripheral-software
criticality: high
goldenPath: web-api
deploymentTarget: azure-appsvc-prod     # ← Azure PaaS (NO K8s required)
secretsProvider: azure-key-vault
observabilityProvider: azure-monitor

config:
  host: canteen-appsvc.dayliff.com
  sku: B1
secrets:
  - canteen-db-password
```

All three use the exact same pipeline spine, same gates, same repository contract, same 24-hour promotion window. Kubernetes is NOT mandatory for any cloud.

☁ Tier 1 — True Multi-Cloud DR (Azure Primary + AWS DR)

```
# catalog/2-business-central/ds-internal--bc-backup.yaml
department: business-central
criticality: critical
goldenPath: backup-system

# Primary: Azure App Service
```

```
deploymentTarget: azure-appsvc-prod

# DR: AWS App Runner (completely independent cloud, NO K8s needed)
drTarget: aws-app-runner-prod

secretsProvider: azure-key-vault   # Primary secrets
drSecretsProvider: aws-secrets-manager  # DR secrets (synced securely)
observabilityProvider: datadog     # 3rd party aggregator for multi-cloud view

resilience:
  ha: true
  dr: true                         # Auto-failover to AWS if Azure fails
  cyberResilience: true            # Immutable backups in both clouds
```

If Azure has a regional outage → Front Door / Global routing → **AWS App Runner takes over automatically**. Your data is replicated, your app is identical, your secrets are safely synced. **No single cloud vendor can take you down.**

Tier 1 — Non-K8s Mobile BaaS (Firebase + GCP Secret Manager)

```
# catalog/1-digital-channels/ds-internal--customer-app.yaml
department: digital-channels
criticality: medium
goldenPath: mobile-baas
deploymentTarget: firebase            # Routes to providers/compute/firebase
secretsProvider: gcp-secret-manager   # Routes to providers/secrets/gcp-secret-manager
observabilityProvider: firebase-analytics

config:
  projectId: ds-customer-app-prod
  services: [auth, firestore, storage]

secrets:
  - firebase-admin-sdk
  - stripe-publishable-key
```

No Kubernetes required. Pipeline uses `firebase deploy` natively.

Tier 2 — Assisted / Specialized

```
# catalog/5-ai-sizing/ds-internal--pump-sizing-training.yaml
department: ai-sizing
criticality: high
goldenPath: ai-ml-training
deploymentTarget: azure-aks-gpu
secretsProvider: azure-key-vault
observabilityProvider: azure-monitor

workload:                             # team-supplied (platform can't default)
  framework: pytorch
  gpuType: nvidia-t4
  gpuCount: 2
  distributed: true
  trainingData: { source: azure-blob, container: pump-sizing-datasets }
  output: { modelRegistry: mlflow, artifactPath: models/pump-sizing/ }
```

Specialized infra (aks-gpu-01) provisioned **on-demand** when this merges.

⍰ Tier 3 — Self-Managed / Escape Hatch (designed mature, code later)

```
# catalog/1-digital-channels/ds-internal--payment-gateway.yaml
department: digital-channels
criticality: high
goldenPath: custom                    # escape hatch

custom:
  pipelineRepo: ds-internal/payment-gateway-pipeline
  compliance: pci-dss
  deploymentTarget: kamatera-vm       # custom VM provider
  secretsProvider: cyberark-conjur    # enterprise secrets
  observabilityProvider: datadog
  contracts:                           # what the team must satisfy
    publishSecurityResults: true
    registerObservability: true
    applyCostTags: true
  reReviewDate: 2027-02-13             # 6-month re-review
```

	Tier 1	Tier 2	Tier 3
Catalog generated by	scaffold (auto)	scaffold (auto)	registration/approval
Repo created?	✓	✓	✗(team owns)
Pipeline binding	app-dispatcher	app-dispatcher	team's own + spine
Dynamic Targets	✓(any provider)	✓(any provider)	✓(any provider)
Approval gate	routine merge	routine merge	**explicit approval**
Build now?	✓	✓	code later

DIAGRAM 7 — Dynamic Target & Provider Registries + Guardrails + Cost

```
ds-platform/target-registry/targets.yml     ◄── fully dynamic, append-only
─────────────────────────────────────────────────────────────────────────

targets:
  # ─── AZURE (K8s & PaaS — Staging, Prod, DR) ───
  azure-aks-staging:    { provider: azure-aks,         serviceConnection: ado-sc-azure-staging, resource: aks-stg-01,    cloud: azure, region: southafricanorth, tier: staging }
  azure-aks-prod:       { provider: azure-aks,         serviceConnection: ado-sc-azure-prod,    resource: aks-prod-01,   cloud: azure, region: southafricanorth, tier: prod }
  azure-aks-dr:         { provider: azure-aks,         serviceConnection: ado-sc-azure-dr,      resource: aks-dr-01,     cloud: azure, region: southafricawest,  tier: dr }
  azure-appsvc-staging:{ provider: azure-app-service, serviceConnection: ado-sc-azure-staging, resource: app-stg-01,    cloud: azure, region: southafricanorth, tier: staging }
  azure-appsvc-prod:    { provider: azure-app-service, serviceConnection: ado-sc-azure-prod,    resource: app-prod-01,   cloud: azure, region: southafricanorth, tier: prod }
  azure-appsvc-dr:      { provider: azure-app-service, serviceConnection: ado-sc-azure-dr,      resource: app-dr-01,     cloud: azure, region: southafricawest,  tier: dr }

  # ─── AWS (K8s & PaaS — Staging, Prod, DR) ───
  aws-eks-staging:      { provider: aws-eks,           serviceConnection: ado-sc-aws-staging,   resource: eks-stg-01,    cloud: aws,   region: eu-west-1,        tier: staging }
  aws-eks-prod:         { provider: aws-eks,           serviceConnection: ado-sc-aws-prod,      resource: eks-prod-01,   cloud: aws,   region: eu-west-1,        tier: prod }
  aws-eks-dr:           { provider: aws-eks,           serviceConnection: ado-sc-aws-dr,        resource: eks-dr-01,     cloud: aws,   region: us-east-1,        tier: dr }
  aws-runner-staging:   { provider: aws-app-runner,    serviceConnection: ado-sc-aws-staging,   resource: run-stg-01,    cloud: aws,   region: eu-west-1,        tier: staging }
  aws-runner-prod:      { provider: aws-app-runner,    serviceConnection: ado-sc-aws-prod,      resource: run-prod-01,   cloud: aws,   region: eu-west-1,        tier: prod }
  aws-runner-dr:        { provider: aws-app-runner,    serviceConnection: ado-sc-aws-dr,        resource: run-dr-01,     cloud: aws,   region: us-east-1,        tier: dr }

  # ─── GCP (K8s & PaaS — Staging, Prod, DR) ───
  gcp-gke-staging:      { provider: gcp-gke,           serviceConnection: ado-sc-gcp-staging,   resource: gke-stg-01,    cloud: gcp,   region: europe-west1,     tier: staging }
  gcp-gke-prod:         { provider: gcp-gke,           serviceConnection: ado-sc-gcp-prod,      resource: gke-prod-01,   cloud: gcp,   region: europe-west1,     tier: prod }
```

```
  gcp-gke-dr:          { provider: gcp-gke,           serviceConnection: ado-sc-gcp-dr,       resource: gke-dr-01,     cloud: gcp,   region: us-central1,      tier: dr }
  gcp-run-staging:     { provider: gcp-cloud-run,     serviceConnection: ado-sc-gcp-staging,  resource: run-stg-01,    cloud: gcp,   region: europe-west1,     tier: staging }
  gcp-run-prod:        { provider: gcp-cloud-run,     serviceConnection: ado-sc-gcp-prod,     resource: run-prod-01,   cloud: gcp,   region: europe-west1,     tier: prod }
  gcp-run-dr:          { provider: gcp-cloud-run,     serviceConnection: ado-sc-gcp-dr,       resource: run-dr-01,     cloud: gcp,   region: us-central1,      tier: dr }

  # ——— DIGITALOCEAN (PaaS & K8s) ———
  do-app-staging:      { provider: digitalocean-app,  serviceConnection: ado-sc-do-staging,   resource: do-app-stg,    cloud: do,    region: nyc1,             tier: staging }
  do-app-prod:         { provider: digitalocean-app,  serviceConnection: ado-sc-do-prod,      resource: do-app-prod,   cloud: do,    region: nyc1,             tier: prod }
  do-doks-dr:          { provider: digitalocean-doks, serviceConnection: ado-sc-do-dr,        resource: do-doks-dr,    cloud: do,    region: sfo2,             tier: dr }
```

Adding a new target = 3 things (not just a line):

```
1. Infrastructure in Target Cloud  ← provisioned via Terraform (providers/compute/<target>/)
2. ADO service connection          ← created once in Azure DevOps
3. Registry entry (PR)             ← appended to targets.yml, validated by targets-ci.yml
```

Guardrails that keep it safe as it grows:

```
┌──────────────────────────────────────────────────────────┐
│ targets-ci.yml (on PR to targets.yml)                    │
│   ✓ schema valid  ✓ unique names  ✓ valid tier/region    │
│   ✓ service connection exists (ADO REST)                 │
│   ✓ target resource exists in Cloud (az/aws/gcloud/doctl)│
│   ✓ no orphaned catalog references on removal            │
├──────────────────────────────────────────────────────────┤
│ catalog-ci.yml cross-check                               │
│   ✓ every app's deploymentTarget: value exists in targets.yml │
│   ✓ every app's secretsProvider: value exists in secrets.yml  │
├──────────────────────────────────────────────────────────┤
│ Platform-only RBAC                                       │
│   ✓ only platform team merges target-registry            │
└──────────────────────────────────────────────────────────┘
```

What About Cost? (Multi-cloud billing handled via dynamic cost tags)

Cloud	Cost Tagging Mechanism
Azure	Resource tags on all Azure resources
AWS	AWS Cost Allocation Tags
GCP	GCP Labels + Billing export to BigQuery

The `platform-policy.yml` automatically applies the right tag schema per cloud, and the **Cloud Engineering Dashboard** aggregates costs across all clouds by department/app.

Target Type decision (§20 cost & complexity):

Isolation / Workload need	Use
Microservices / complex networking	**Kubernetes (AKS/EKS/GKE/DOKS)**
Standard Web Apps / APIs (No K8s overhead)	**PaaS (App Service, App Runner, Cloud Run, DO App)**
Mobile backend / Auth / Realtime DB	**BaaS (Firebase/Supabase/Appwrite)**
Event-driven / lightweight tasks	**Serverless (Lambda, Cloud Functions, Azure Functions)**
Static sites / CMS / simple backups	**VMs (DigitalOcean/Hostinger/Kamatera)**

DIAGRAM 8 — Concurrency (The Two Scenarios)

Scenario 1: 10 PRs on the SAME repo (`ds-internal/canteen`)

```
PR #1 … PR #10 ──► 10 INDEPENDENT validation runs (parallel, limited by agents)
   │
   │   EACH run (triggered on feature branch):
   │    • repo-contract-check (reads repo — no contention)
   │    • reads the SAME catalog entry (read-only → no contention)
   │    • builds/tests its own feature branch
   │    • publishes results tagged by PR/build id (no collision)
   │    • builds artifact with UNIQUE tag (no registry overwrite)
   │    • deploys to ISOLATED ephemeral preview environment (validate mode)
   │    • ⚠ none touch shared staging/prod · DR never triggers (prod-only)
   │
   ▼
MERGE into develop branch is SEQUENTIAL (git advances one merge at a time)
   ├─ PRs merge one at a time (Squash & Merge enforced by branch policy)
   ├─ if PR #3 & #7 touch same file → later one conflicts → rebase
   ├─ branch policy on develop branch: build validation + reviewers before merge
   └─ concurrency: cancel-in-progress-runs=true
        → only newest develop branch build proceeds (avoids deploy storm)
   │
   ▼
Promotion to production:
   ├─ Pipeline pauses at staging for 24-hour approval window
   ├─ Upon approval, pipeline creates ADO main branch, merges code (Create a merge commit), deploys to prod
   └─ approval is manual (never auto-approved)
```

Safe because: validate mode + read-only catalog + unique tags + concurrency controls + strict branch protections.

Scenario 2: 10 PRs across DIFFERENT repos/projects

```
ds-internal/canteen ──────────► run 1 → own catalog entry
ds-internal/asset-register ──► run 2 → own catalog entry
ds-external/hack-2026-team-1 ► run 3 → own catalog entry
… (10 repos)                    …
   │
   ▼
10 FULLY INDEPENDENT, PARALLEL runs
   • each reads its OWN catalog entry (no shared file)
   • each routes to its OWN goldenPath + template library
   • each targets its OWN deployment target (AKS, App Runner, Firebase, DO, etc.)
   • each publishes its OWN results to Grafana
   • each uses its OWN secrets provider namespace
   • no shared branch, environment, or catalog
   │
   ▼
ONLY LIMITER: AGENT POOL CAPACITY + TARGET API RATE LIMITS
   (fewer agents than runs → extras queue, then run)
```

Safe because: no coupling at all — just agent capacity and API limits.

DIAGRAM 9 — Template Selection + Wrong-Template Handling

How the developer chooses (no detection → guidance must be excellent)

```
Onboarding presents GOLDEN PATH CATALOG (_catalog.json)
   Each card shows: What / Use when / Don't use when / Examples / You get
   │
   ▼
DECISION TREE (fast path)
   What are you building?
   ├─ User-facing web app or API? ──────────► web-api (AKS / EKS / GKE / App Runner / Cloud Run / App Service)
   ├─ Mobile app backend (Auth/DB)? ────────► mobile-baas (Firebase / Supabase)
   ├─ Serve an ML model as an API? ─────────► ai-ml           (Tier 1)
   ├─ TRAIN an ML model (GPUs)? ────────────► ai-ml-training  (Tier 2)
   ├─ Report / dashboard / data pipeline? ──► data-bi
   ├─ Business Central extension / AL? ─────► business-central
   ├─ Multi-cloud DR backup? ───────────────► backup-system (DigitalOcean / AWS)
   ├─ Static site / CMS / WordPress? ───────► static-site (Hostinger / SiteGround)
   ├─ Azure infra / Terraform? ─────────────► infrastructure
   ├─ App around Business Central? ─────────► peripheral-software
   ├─ Streaming / edge (dedicated infra)? ──► streaming / edge (Tier 2)
   └─ Doesn't fit any? ─────────────────────► Tier 3 escape hatch
```

Standard vs. Specialized — how you tell

```
meta.yaml per template:
  web-api:        { tier: 1, type: standard,    requiresDedicatedInfra: false }
  mobile-baas:    { tier: 1, type: standard,    requiresDedicatedInfra: false, usesK8s: false }
  ai-ml-training: { tier: 2, type: specialized, requiresDedicatedInfra: true,
                    provisionOnDemand: true, teamSuppliesConfig: [gpuType,…] }

requiresDedicatedInfra: true  = specialized (Tier 2)
usesK8s: false                = BaaS / PaaS / Serverless target
```

Wrong template / needs improvement

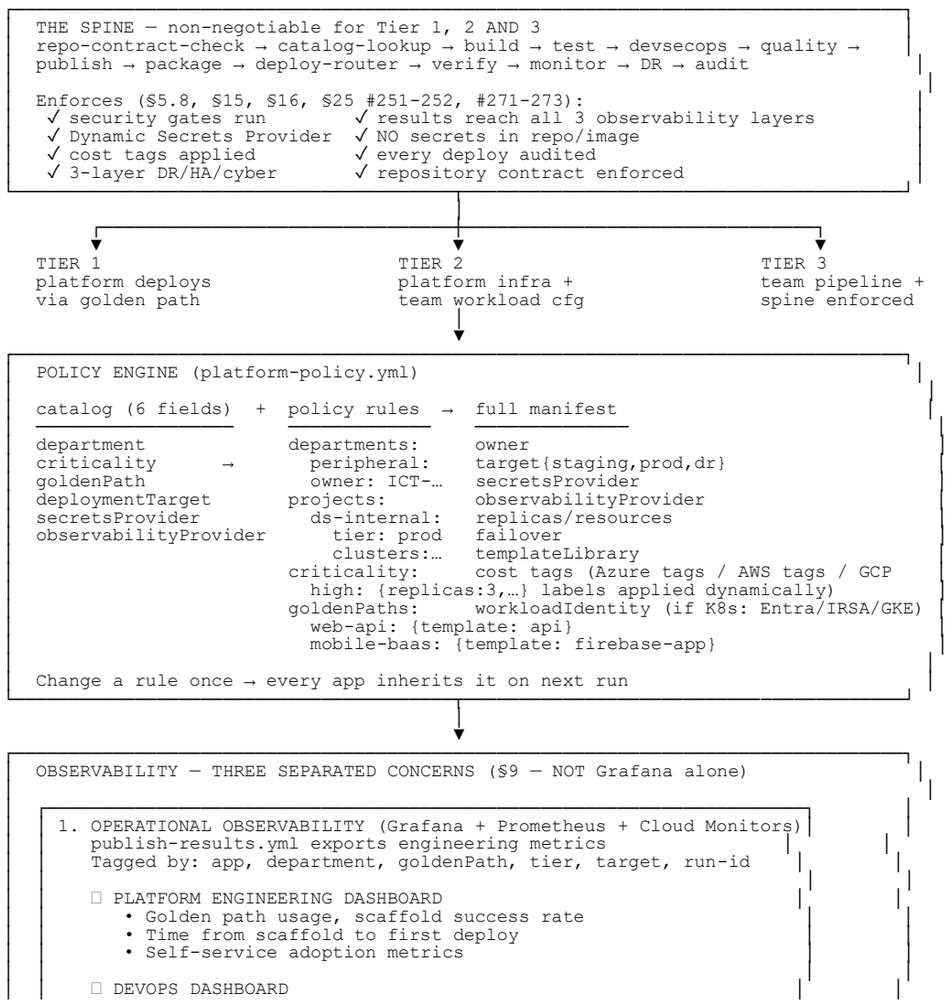
```
WRONG, realised EARLY (before 1st deploy)
   → re-run scaffold, pick correct path, old entry replaced (PR)

WRONG, realised LATE (build fails / runtime off)
   → build error names the mismatch ($258)
   → "Switch Golden Path" action updates catalog goldenPath via PR

RIGHT, but THIS APP needs more
   → add app-level override in the catalog entry's config:
```

```
RIGHT, but the TEMPLATE itself needs improvement
   → PR to golden-path-templates → platform reviews
   → template VERSION bumps (semver) → apps OPT IN
```

DIAGRAM 10 — The Spine + Policy Engine + Separated Observability (All 7 Disciplines)



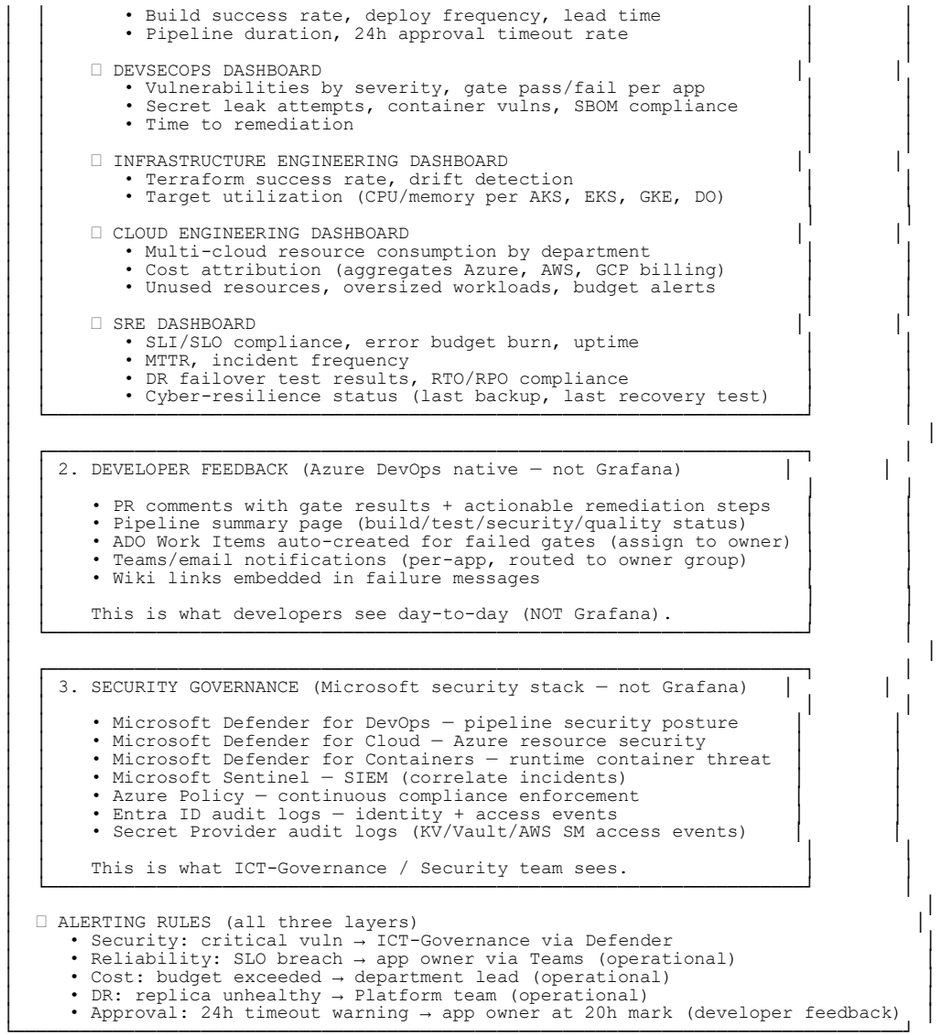


DIAGRAM 11 — Build Phasing (Tier 1 → Tier 2 → Tier 3 + Portal Evolution + Multi-Cloud)

PHASE 0 — Landing zone

Terraform: subscription, VNet, ACR, Key Vault, Entra groups
AKS clusters: aks-test-01, aks-prod-01, aks-dr-01, aks-sandbox-01
Service connections: ado-sc-repo-creator, ado-sc-github-sync, gh-sc-hackathon-setup
Workload Identity infrastructure: OIDC issuer enabled on all AKS clusters

PHASE 1 — Tier 1 core + External flow ◄── build first (PowerShell era)
Internal onboarding: onboard-app.ps1 + Terraform (ado-repo module)
External onboarding: onboard-hackathon.ps1 + Terraform (github-repo + ado-repo modules)
golden-path-templates (web-api first) + _catalog.json + decision tree
platform-policy.yml + catalog-schema.json + catalog-ci.yml
app-dispatcher.yml + spine (repo-contract-check → ci-base → gates → package → deploy-router)
Workload Identity + Key Vault integration (default for K8s apps)
target-registry + targets-ci.yml (with staging/prod/dr environments mapped)
Strict Git Workflows: Branch protections, Squash & Merge to develop, Merge Commit to main
24-hour promotion window (Pipeline creates ADO main branch post-approval → deploys prod)
One-way sync service (GitHub main branch → ADO develop branch)
Repository contract: required/allowed/forbidden enforced
PILOT: scaffold internal app (canteen) + onboard hackathon team (hack-2026-team-1)

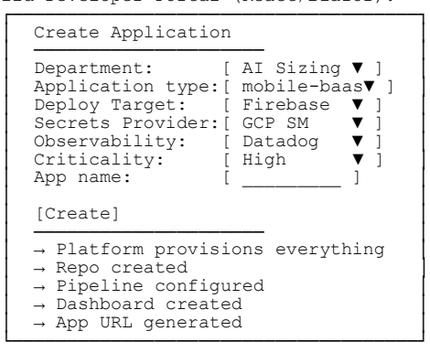
PHASE 2 — DevSecOps + observability hardening
full gate suite · publish-results → 3 observability layers · alerting
All 7 discipline dashboards in Grafana (operational)
ADO PR comments + Work Items (developer feedback)
Microsoft Defender for DevOps/Cloud/Containers + Sentinel (governance)

PHASE 3 — SRE + DR + cyber-resilience
monitor-register · SLI/SLO · dr-provision (HA + DR + cyber-resilience) · dr-teardown
Immutable backups · quarterly recovery tests · Global routing failover

PHASE 4 — Expand Providers (Multi-Cloud, BaaS & PaaS) ◄── DYNAMIC EXPANSION
PHASE 4a: AWS EKS + App Runner + AWS Secrets Manager (Add when you have AWS workloads)
PHASE 4b: GCP GKE + Cloud Run + GCP Secret Manager (Add for GCP-specific services)
PHASE 4c: Multi-cloud DR (Azure ↔ AWS ↔ GCP failover routing)
Add providers/compute/digitalocean-app (for BC backup DR scenario)
Add providers/compute/firebase & supabase (for mobile-baas Golden Path)
Add providers/secrets/hashicorp-vault (enterprise secrets)
Add providers/compute/kamatera-vm & hostinger (for legacy/static sites)
Remaining Tier 1 paths + Tier 2 framework
specialized/ templates + on-demand infra (aks-gpu-01 etc.)

PHASE 5 — Tier 3 escape hatch ◄── code later
register-custom-app.ps1 + escape-hatch.yml + contract-verify.yml
approval workflow · dedicated/isolated cluster · re-review
(built only if Tier 3 demand materializes)

PHASE 6 — Platform API + Developer Portal ◄── end-state (post-Phase 3)
Expose scaffold/catalog/target-registry as REST API
Build Developer Portal (React/Blazor):



PowerShell scripts kept as fallback / CI automation
This is when it becomes a true INTERNAL DEVELOPER PLATFORM

Summary — Everything Captured (Ultimate God Mode + Dynamic Providers + Multi-Cloud)

Topic	Where it lives
Very beginning = onboarding PowerShell scripts (internal + external)	Diagrams 3, 4
Service connections create repos (Basic license users can't)	Diagrams 2, 3, 4
Internal scaffold (no detection)	Diagram 3
External hackathon onboarding (GitHub + ADO, one-way sync)	Diagram 4
Catalog generated by scaffold (T1/T2) / registration (T3)	Diagrams 3, 4, 6
Catalog lives in app-catalog, not app repo	Diagrams 2, 5
Repository contract (required/allowed/forbidden)	Diagrams 2, 5
git push → production flow (internal + external)	Diagram 5
24-hour approval window (auto-CANCEL if not approved — never auto-approve)	Diagram 5
ADO main branch created dynamically by pipeline post-approval (Merge Commit)	Diagrams 1, 2, 3, 4, 5
Strict Internal Workflow: Feature branch → PR to develop (Squash & Merge)	Diagrams 1, 2, 3, 5, 8
Strict External Workflow: Feature branch → PR to GH develop (Squash) → Lead PRs to GH main (Merge Commit)	Diagrams 1, 2, 4
Auto-delete head branches enabled in GitHub	Diagram 4
Branch Protections (no direct pushes to develop/main in either system)	Diagrams 1, 2, 3, 4
ADO develop protection bypassed ONLY by sync service (external) or PRs (internal)	Diagrams 2, 4
GitHub main branch created at scaffold (for external PR workflows)	Diagrams 1, 2, 4
One-way sync service (GitHub → ADO, never back)	Diagram 4
Dynamic Deployment Target Registry + targets-ci + guardrails	Diagram 7
Target vs Namespace vs BaaS decision (cost §20)	Diagram 7
Multi-Cloud Cost Tagging (Azure tags, AWS tags, GCP labels)	Diagram 7
Concurrency: same repo / different repos	Diagram 8
3 mature tiers (T1 & T2 now, T3 code later)	Diagrams 2, 6, 11
Policy engine + universal spine	Diagram 10
Observability in 3 separated layers (operational/developer/governance)	Diagram 10
Dynamic Secrets Provider Abstraction (Azure KV, HashiCorp, AWS SM, etc.)	Diagrams 5, 6, 10
Dynamic Observability Provider (Azure Monitor, CloudWatch, Cloud Logging)	Diagrams 1, 5, 6, 10
Multi-Cloud Routing Mechanics (Same Spine, Provider Plugins)	Diagram 5
Concrete Catalog Examples (Same App across Azure, AWS, GCP)	Diagram 6
Provider Plugin Architecture (Folder structure for EKS/GKE/PaaS)	Diagram 2
Security Model per Cloud (Workload Identity / IRSA / GKE WI / Managed ID)	Diagram 5
Multi-Cloud DR Scenario (Azure Primary → AWS DR)	Diagram 6
Non-K8s workloads supported (Firebase, Supabase, Amplify, PaaS, VMs)	Diagrams 1, 5, 6, 9
Modular Provider Plugin Architecture (Extensibility)	Diagram 2, 11
DR expanded: HA + DR + cyber-resilience (immutable backups)	Diagrams 5, 11

Template selection + wrong-template handling	Diagram 9
DR / failover / teardown	Diagram 5
Developer Portal as end-state (Phase 6)	Diagram 11
All 7 engineering disciplines publish results	Diagram 10
Staging, Prod, DR mapped for ALL targets in Registry	Diagram 7
Kubernetes is strictly optional per cloud (PaaS/Serverless supported)	Diagrams 1, 2, 5, 6, 7
Dynamic Hackathon Inputs (Variable team sizes & per-team target choices)	Diagram 4

Final Corrections Checklist (All 16 Expert Review Points Addressed)

1. ✅**Branch model (ADO)**: `develop` branch created at scaffold and PROTECTED. `main` branch created *dynamically by the pipeline* upon staging approval. Pipeline merges `develop` to `main` using **Create a merge commit**, then deploys the verified staging image to PROD.

2. ✅**24-hour window, NOT auto-approval**: explicitly called "24-hour approval window with automatic cancellation on timeout" — timeout NEVER means approval

3. ✅**Repository contract** instead of strict "src/+tests/ only": required / allowed / forbidden lists, allows docs/migrations/language manifests, forbids pipeline files/secrets/.env/k8s manifests/Helm

4. ✅**Dynamic Secrets Provider Abstraction**: catalog declares `secretsProvider` (Azure KV, HashiCorp, AWS SM, CyberArk, Akeyless). Platform routes to correct provider module

5. ✅**DR = 3 layers**: High Availability (replicas/zones/HPA) + Disaster Recovery (secondary region/geo-replication/Global routing) + Cyber-resilience (immutable backups/WORM/isolated access/quarterly recovery tests)

6. ✅**Developer portal evolution path**: PowerShell (Phase 1) → Platform API (Phase 2) → Developer Portal UI (Phase 6)

7. ✅**Observability NOT Grafana alone**: explicitly separated into Operational (Grafana+Prometheus+Cloud Monitors), Developer feedback (ADO PR/Work Items/Teams), Security governance (Defender+Sentinel+Policy+Entra+KV audit logs)

8. ✅**Dynamic Deployment Target Abstraction**: catalog declares `deploymentTarget` (AKS, EKS, GKE, DigitalOcean, Firebase, Supabase, AWS Amplify, Kamatera, Hostinger, etc.). Supports K8s and non-K8s

9. ✅**Provider-agnostic deploy step**: Pipeline routes to native CLIs/APIs (firebase deploy, supabase db push, doctl) based on target. Mobile apps and static sites fully supported

10. ✅**Modular Provider Plugin Architecture**: New tech stack = add folder to `providers/compute/` or `providers/secrets/`. Core Spine never changes. Freedom of choice for any tech stack

11. ✅**Branch model (GitHub)**: External onboarding creates *both* protected `develop` and `main` branches in GitHub at scaffold time. Students use feature branches, **Squash & Merge** to `develop`. Student Lead uses **Create a merge commit** from `develop` to `main`. Head branches auto-delete. ADO `develop` is strictly protected; only the sync service can write to it.

12. ✅**All 11 diagrams + summary section included**: nothing omitted

13. ✅**Multi-Cloud explicitly detailed & fused**: AWS EKS, GCP GKE, IRSA, Workload Identity, Cross-Cloud DR, Dynamic Observability, and Multi-Cloud Cost Tagging fully integrated into the core diagrams and examples without breaking the architecture flow.

14. ✅**Environment Parity & K8s Optional**: Staging, Prod, and DR are explicitly mapped for ALL targets. Kubernetes is strictly optional — AWS App Runner, GCP Cloud Run, and Azure App Service are fully supported alongside their K8s counterparts via identical pipeline spines.

15. ✅**Dynamic Hackathon Inputs**: Team member counts are dynamic per team (e.g. 3, 4, 6 members). Target, secrets provider, and observability provider are selected *per team* during onboarding, rather than applying a single global default to all teams.

16. ✅**Strict Git Merge Strategies**: Explicitly enforced: **Squash & Merge** for feature → `develop`. **Create a merge commit** for `develop` → `main` (both GitHub Lead and ADO Pipeline). **Auto-delete head branches** enabled in GitHub. No direct pushes allowed to protected branches.

Build Order (Your Phase 1 First Deliverables)

The first concrete deliverables that prove the platform works end-to-end:

```
1. onboard-app.ps1          (internal scaffold — PowerShell + Terraform)
2. onboard-hackathon.ps1    (external scaffold — GitHub + ADO + sync + dynamic team arrays)
3. Terraform: ado-repo + github-repo modules (service-connection-backed + branch protections)
4. web-api/template.json    (first Golden Path)
5. _catalog.json            (front door decision tree)
6. platform-policy.yml      (derivation brain)
7. catalog-schema.json + catalog-ci.yml (validation)
8. app-dispatcher.yml + _shared/*.yml (the spine + deploy-router + promote-to-main)
9. target-registry/targets.yml + targets-ci.yml (staging/prod/dr mapping)
10. providers/compute/azure-aks/ + providers/secrets/azure-key-vault/ (first plugins)
11. 24-hour promotion gate + pipeline-driven ADO main branch creation + one-way sync service
12. Grafana operational dashboards (Phase 2)
13. Microsoft Defender + Sentinel integration (Phase 2)
14. providers/compute/aws-eks/ + providers/compute/aws-app-runner/ + providers/secrets/aws-secrets-manager/ (Phase 4 plugins)
15. providers/compute/gcp-gke/ + providers/compute/gcp-cloud-run/ + providers/secrets/gcp-secret-manager/ (Phase 4 plugins)
```

Pilot: scaffold `canteen` (internal) + `hack-2026-team-1` (external) end-to-end through this platform.

This is the **Ultimate God Mode (Dynamic, Modular & Multi-Cloud)** version. Ready to generate the Phase-1 starter files as concrete, ready-to-commit code next?